Sheet 02 - Multilayer Perceptrons

Introduction to Deep Learning - Summer Semester 2026

Ulf Krumnack & Robin Rawiel - Universität Osnabrück

Due: April 26, 2026

Task 1: Theory Questions [6 points]

1.1 Activations & Nonlinearity [1 point]

1. Write down the formulas for the **sigmoid** $\sigma(z)$ and **ReLU** activation functions. What is the output range of each?
2. Can a neural network with only linear activation functions learn non-linear decision boundaries? Explain why or why not.

1.2 Computational Graphs & Backpropagation [2 points]

1. *(1 point)* Draw the **computational graph** for a one-hidden-layer MLP ($x \to z^{(1)} \to a^{(1)} \to z^{(2)} \to \hat{y}$) in explicit style, where you show both value nodes and operation nodes. Label the dimensions of the weight matrices and activation vectors for an input of size d, a hidden layer of size h, and a single output.
2. *(1 point)* In the backward pass, the error signal $\delta^{(\ell)}$ is propagated from layer ℓ to layer $\ell - 1$ via the recursion

$$\delta^{(\ell-1)} = \left(W^{(\ell)}\right)^\top \delta^{(\ell)} \odot \varphi'\left(z^{(\ell-1)}\right)$$

and the weight gradient is computed as

$$\frac{\partial \mathcal{L}}{\partial W^{(\ell)}} = \delta^{(\ell)} \left(a^{(\ell-1)}\right)^\top.$$

Explain **in words** the role of each component: (a) the transpose $(W^{(\ell)})^\top$, (b) the element-wise product $\odot \varphi'(z^{(\ell-1)})$, and (c) the outer product $\delta^{(\ell)}(a^{(\ell-1)})^\top$.

1.3 Regularization [2 points]

1. *(0.5 points)* How does **early stopping** work? What pattern in the learning curves (training loss vs. validation loss) tells you when to stop?
2. *(0.5 points)* How does **dropout** work during training vs. during inference?
3. *(1 point)* Write the **L2-regularized loss** function. How does the regularization parameter λ affect the weights during training?

1.4 Mini-batch Learning [1 point]

1. *(0.5 points)* Explain the difference between **batch gradient descent**, **mini-batch gradient descent**, and **stochastic gradient descent (SGD)**.
2. *(0.5 points)* Define the term **epoch**. If you have 10,000 training examples and use a mini-batch size of 250, how many parameter updates are performed per epoch?

Task 2: MLP from Scratch [8 points]

Learning objectives:

- Implement forward and backward pass manually in NumPy
- Understand backpropagation mechanics
- Optional: verify correctness via gradient checking

2.1 Activation Functions [1 point]

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split

np.random.seed(42)


def relu(z):
    # TODO: Your solution here


def relu_derivative(z):
    # TODO: Your solution here


def sigmoid(z):
    # TODO: Your solution here


def sigmoid_derivative(z):
    # TODO: Your solution here
```

2.2 MLP Class [3 points]

Implement a multi-layer perceptron with configurable layer sizes, small random weight initialization, ReLU hidden activations, and sigmoid output.

```
class MLP:
    def __init__(self, layer_sizes):
        """Initialize MLP with given layer sizes, e.g. [2, 32, 16, 1].

        Use small random initialization (scale 0.1) for weights and zeros for biases.
        """
        # TODO: Your solution here

    def forward(self, X):
        """Forward pass. Store intermediate values for backprop.

        Args:
            X: Input, shape (N, D).

        Returns:
            Output, shape (N, 1).
        """
        # TODO: Your solution here
```

```
    def compute_loss(self, y_true, y_pred):
        """Binary cross-entropy loss."""
        # TODO: Your solution here

    def backward(self, y_true):
        """Backward pass. Compute gradients for all weights and biases.

        Args:
            y_true: True labels, shape (N, 1).

        Returns:
            dW: List of weight gradients.
            db: List of bias gradients.
        """
        # TODO: Your solution here

    def train_step(self, X, y, lr=0.1):
        """One forward + backward pass with parameter update."""
        # TODO: Your solution here
```

2.3 Train on Moons Dataset [2 points]

```
X, y = make_moons(n_samples=500, noise=0.2, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
y_train = y_train.reshape(-1, 1)
y_test = y_test.reshape(-1, 1)
```

```
# TODO: Your solution here
```

2.4 Visualize Decision Boundary [1 point]

```
def plot_decision_boundary(model, X, y):
    """Plot the decision boundary of the MLP."""
    # TODO: Your solution here
```

```
# TODO: Your solution here
```

2.5 Gradient Checking (Bonus) [1 point]

Verify your backpropagation implementation by comparing analytical gradients with numerical approximations using $\epsilon = 10^{-7}$.

```
def gradient_check(model, X, y, epsilon=1e-7):
    """Compare analytical and numerical gradients."""
    # TODO: Your solution here
```

```
# TODO: Your solution here
```

Task 3: MLP with PyTorch [6 points]

Learning objectives:

- Learn the PyTorch API (`nn.Module`, `optim`, `DataLoader`)
- Compare different regularization methods
- Understand the effect of network depth and width

3.1 Setup and Model [2 points]

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split

torch.manual_seed(42)
np.random.seed(42)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# Data
X, y = make_moons(n_samples=1000, noise=0.2, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

X_train_t = torch.FloatTensor(X_train).to(device)
y_train_t = torch.FloatTensor(y_train).unsqueeze(1).to(device)
X_test_t = torch.FloatTensor(X_test).to(device)
y_test_t = torch.FloatTensor(y_test).unsqueeze(1).to(device)
train_dataset = TensorDataset(X_train_t, y_train_t)
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
```

```
class MLP(nn.Module):
    def __init__(self, input_dim=2, hidden_dims=[128, 128, 128], output_dim=1, dropout=0.0):
        """MLP with configurable hidden layers and optional dropout.

        Args:
            input_dim: Number of input features.
            hidden_dims: List of hidden layer sizes.
            output_dim: Number of outputs.
            dropout: Dropout probability (0 = no dropout).
        """
        super().__init__()
        # TODO: Your solution here

    def forward(self, x):
        # TODO: Your solution here
```

3.2 Training Function [2 points]

```
def train_model(model, train_loader, X_test_t, y_test_t, lr=0.001, weight_decay=0.0, epochs=200):
    """Train the model and track metrics.

    Args:
        model: PyTorch model.
        train_loader: DataLoader for training data.
        X_test_t, y_test_t: Test tensors.
        lr: Learning rate.
        weight_decay: L2 regularization (passed to optimizer).
        epochs: Number of epochs.
```

```
    Returns:
        Dictionary with train_losses, test_losses, train_accs, test_accs.
    """
    # TODO: Your solution here
```

3.3 Regularization Comparison [1 point]

Train five configurations and compare:

1. **Baseline:** `[128, 128, 128]`, no regularization
2. **L2:** weight_decay = 0.01
3. **Dropout:** dropout = 0.3
4. **L2 + Dropout:** both
5. **Small network:** `[16]`

```
# TODO: Your solution here
```

3.4 Depth vs. Width [1 point]

Compare three architectures with roughly the same total parameters:

- **Wide & Shallow:** `[256]`
- **Medium:** `[64, 64]`
- **Deep & Narrow:** `[16, 16, 16, 16, 16]`

```
# TODO: Your solution here
```

3.5 Reflection

1. How does the PyTorch implementation compare to your NumPy version in terms of code complexity and training speed?
2. Which regularization approach worked best? Why?
3. What tradeoffs do you observe between deep-narrow and wide-shallow architectures?